

Guion: Práctica 2 Programación Orientada a Objetos

Presentador: Bienvenidos a la explicación de la Práctica 2 de Programación Orientada a Objetos. Cuando se inicia en un nuevo lenguaje de programación, no se sabe cómo emplear correctamente las estructuras de control condicionales y cíclicas, además se encuentra dificultad en el uso correcto de variables, causando retrasos con la resolución de problemas. Al mismo tiempo, se solicita apoyo a los LLM para diseñar distintas aplicaciones, pero las respuestas generadas suelen dificultar la comprensión del código propuesto. Por eso, esta práctica tuvo dos objetivos: verificar lo que recomienda un LLM acerca del programa de Estudiantes Aprobados y Reprobados, para darnos ideas de cómo realizarlo y comenzar a discernir lo que es útil, y elaborar programas donde se usen estructuras cíclicas, para lograr su entendimiento y emplearlas correctamente en futuros programas. Empecemos por el marco teórico.

Orador 1: El método `main()` en Java es estático y un punto de entrada de la Máquina Virtual de Java al programa. La palabra clave `static` es un modificador que se utiliza para métodos y atributos que permite acceder a ellos sin necesidad de crear un objeto de la clase. El método `main()` es estático para evitar ambigüedad sobre qué constructor debe invocarse.

Por otro lado, la clase `Scanner` se usa para obtener la entrada del usuario y se encuentra en el paquete `java.util`; para usarla se crea un objeto de la clase y se emplean sus métodos disponibles, como `nextLine()` para leer cadenas de texto.

Presentador: Además del método `main` y de `Scanner`, revisamos las estructuras de control que permiten alterar el orden de ejecución de las instrucciones.

Orador 2: Vimos cinco estructuras:

1. `if` evalúa una condición y ejecuta un bloque de instrucciones si esta es verdadera.
2. `for` crea un bucle que se repite un número un número que es conocido de antemano por nosotros los programadores.
3. `mientras` que `while` mantiene el bucle un número de veces desconocido por los programadores mientras la condición especificada sea verdadera.
4. `do-while` evalúa la condición evalúa después de la primera ejecución, garantizando que el bloque se ejecute al menos una vez, a diferencia de `while`.
5. Por último, la estructura `switch` selecciona un bloque de instrucciones según el valor de su sentencia. Es opcional el uso de `break` y `default`.

El ejercicio consistía en preguntar una aplicación de control de aprobados y reprobados.

En el primer programa se repiten dos ciclos el número de alumnos haya y se ponen los contadores en 0, aprobados y reprobados, y se pide al usuario un valor de 1 si aprobó o 2 si reprobó. Al terminar, se muestra el total de aprobados y reprobados, y: si aprobados es cero, todos reprobaron y se muestra un mensaje

especial sin bonificación; si reprobados es cero o es uno, se otorga bonificación; y si reprobados es dos o más, no se otorga bonificación. En el segundo prompt se fijó el número de alumnos en diez,, se usa if-else para validar si el resultado es 1 o 2, y al final una condición if para evaluar si aprobados es mayor o igual a nueve e imprimir el mensaje de bono,

diferencia: uno dio el algoritmo paso a paso con un enfoque más didáctico y el otro LLM con un enfoque más técnico; además, un modelo dejó que el usuario ingresara el número de estudiantes mientras que otro lo fijó en diez,

Presentador: El segundo ejercicio fue el Ejercicio Comparar, donde se trabajó con estructuras cíclicas para determinar el valor máximo entre diez números enteros, manteniendo la restricción de resolverlo usando únicamente tres variables: contador, número y mayor.

Orador 4: El resultado se centra en el método estático main, un punto de entrada ejecutado directamente por la Máquina Virtual de Java que no requiere la creación de una clase previa. A través de la importación de la clase utilitaria Scanner del paquete java.util, se gestiona la captura de datos desde la consola. La lógica de iteración se fundamenta en una estructura while, la cual controla diez lecturas consecutivas mediante una variable de control actualizada con el operador contador++. Tras esto, se emplean estructuras de selección simple if, con el operador num *=-1 para que los valores negativos tengan magnitud positiva, y se compara cada entrada con una variable para determinar el valor máximo. Después, la salida en consola utiliza el operador de concatenación (+) para convertir los números enteros a cadenas de texto. Finalmente, con la función scan.close() evitamos que datos indeseados sean introducidos.

Presentador: El tercer ejercicio fue el Ejercicio Tabla, donde se generó una tabla de valores mostrando la relación entre un número n y sus múltiplos: n, 10n, 100n y 1000n.

Orador 5: Al igual que en el ejercicio anterior, nos centramos en el método estático main. Declaramos una variable de tipo primitivo entero que actúa como variable de control para una estructura while que delimita cinco iteraciones mediante la expresión (i menor que 6) y el operador i++. A través del flujo de salida estándar System.out.println, se imprime una tabla de potencias de diez utilizando sangrías horizontales para alinear las columnas con el tabulador. Por último, gracias a la jerarquía de operaciones, se realizan primero las multiplicaciones entre i y la potencia de diez correspondiente, y luego, con el signo de suma, se concatenan los resultados en una sola cadena de texto.

Presentador: Y así lo comprobamos en las pruebas de validación. En el Ejercicio Comparar, al ingresar los diez números —24, 42, 63, 99, 3, 34, 65, 12, 52 y 73—, el programa mostró correctamente que el número mayor fue 99. En el Ejercicio Tabla, el programa imprimió los encabezados n, n10, n100 y n*1000, y desplegó los resultados del 1 al 5 con sus múltiplos correspondientes, alineados en columnas.

Presentador: Para cerrar, el primer objetivo de la práctica se cumplió, ya que se logró comprender de mejor manera el funcionamiento de un programa con ayuda de la LLM, se pudo comparar lo que decían los distintos modelos, y el alumnado pudo discernir entre lo que podría ser útil y lo que no. El segundo objetivo también se cumplió, ya que se logró entender el funcionamiento de las estructuras cíclicas en el lenguaje de Java, ayudando así a la simplificación y optimización de los programas, evitando la repetición de código.

Presentador: Gracias por acompañarnos en esta explicación de la Práctica 2.